

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D034.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 034 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

Pandas Series/ DataFrame

// Pandas Core

KEY CONCEPTS · 三大要点

- ▶ 索引
- ▶ 对齐
- ▶ 数据类型

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

今天进入 Pandas 的世界。如果说昨天的 NumPy 是处理「纯数字矩阵」的引擎,那 Pandas 就是给这台引擎装上了仪表盘和方向盘 — 它让数据带上了「标签」。什么意思?昨天的数据是一堆光秃秃的数字,你要取某个值得记住它在第几行第几列。而真实的金融数据,每一行是一个日期、每一列是一只股票的名字,这些日期和名字就是「标签」。Pandas 的两大主角 Series 和 DataFrame,就是「带标签的智能表格」。Series 是一列带标签的数据,DataFrame 是一整张带行列标签的表。这一节做五件事 — 第一,讲清楚 Series 和 DataFrame 到底是什么,跟 NumPy 数组什么关系;第二,讲 Pandas 最神的「自动对齐」,两张日期不完全一样的表怎么自动按日期对上;第三,讲 loc 和 iloc 这对最容易搞混的兄弟,一个按标签取、一个按位置取;第四,讲 dtype 数据类型怎么决定你的代码快慢和吃多少内存;第五,讲 apply 这个「批量套函数」的工具。为了让一切落地,我们用真实的沪深 300 成分股,亲手造一张上百只股票的行情大表,所有操作都在这张真表上演示。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 搞懂 Series 和 DataFrame 是什么 — Series 是带标签的一列、DataFrame 是带行列标签的整张表,本质是 NumPy 数组加上了索引
- 02 掌握 Pandas 最强的自动对齐 — 两张日期不完全一样的表做运算,Pandas 自动按索引对齐,缺口处填 NaN,从此告别手工对齐
- 03 分清 loc 和 iloc — loc 按标签(日期、列名)选,iloc 按位置(第几行第几列)选;loc 切片含右端点、iloc 不含,这是高频 bug 来源
- 04 理解 dtype 的重要性 — 列的数据类型决定运算速度和内存占用,float64 跟 float32 内存差一倍,object 类型慢得惊人
- 05 学会 apply 批量套函数 — 对每只股票自动套用一个自定义函数算因子,同时明白 apply 慢、能矢量化就别用 apply

02 · CONTEXT · 历史与背景

小张用 Excel 手工对齐两只股票,算出离谱的相关系数

// 不读历史的策略走不远

讲一个发生在无数散户身上的真实糗事。小张想验证一个想法:茅台和五粮液这两只白酒龙头,是不是高度同步?他打算算一下两者的相关系数。

他不会编程,用的是 Excel。他从行情软件分别导出茅台和五粮液过去一年的每日收盘价,粘贴到 Excel 的两列里,然后用相关系数函数一拉,得到一个数。结果这个数低得离谱,只有零点三,跟他「两只白酒龙头应该高度同步」的直觉完全不符。他百思不得其解,以为是自己看走眼了。

问题出在哪?出在「对齐」上。茅台和五粮液虽然都是 A 股,但某些天可能一只停牌、一只正常交易,或者小张导出数据时一只多了一行、一只少了一行。两列数据从某一行开始,日期就错位了——第 100 行,茅台对应的是 3 月 10 号,五粮液对应的却是 3 月 11 号。再往后全部串位。用错位的数据算相关系数,结果当然是垃圾。这就是著名的「一行错,行行错」。

小张如果用 Pandas,这个问题压根不会发生。因为 Pandas 的每一列数据都自带「日期标签」,做任何运算时,它会先看标签,自动把同一个日期的数据对齐,日期对不上的地方就标成 NaN(缺失),绝不会发生张冠李戴的错位。这个能力叫「自动对齐」,是 Pandas 区别于 Excel 和 NumPy 最核心、最强大的特性。

这个故事的教训是——在金融数据里,「对齐」是一等一的大事。多少看似神秘的 bug、多少离谱的回测结果,根子都在日期没对齐。Excel 靠人眼对齐,迟早出错;Pandas 靠索引自动对齐,从根上杜绝。今天我们就用真实的上百只沪深 300 成分股,亲手感受自动对齐的威力,顺便把 Series、DataFrame、loc、iloc、dtype、apply 这套量化日常最高频的工具一次学透。学完这一节,你处理数据的方式会从「Excel 手工党」彻底升级成「专业数据党」。

03 · CORE CONCEPTS · 核心概念详解

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

Series 和 DataFrame — 给 NumPy 数组贴上标签

先建立最核心的心智模型。Series 是「一列带标签的数据」。你可以把它想象成 Excel 里的一竖列,但每个格子前面还跟着一个标签。比如茅台一年的收盘价,值是一串价格数字,标签是对应的日期。Series 本质上就是一个 NumPy 数组,外加一个叫「索引(index)」的标签数组配套。

DataFrame 则是「一整张带行列标签的表」。你可以把它想象成整个 Excel 工作表:每一行有个行标签(通常是日期),每一列有个列标签(通常是股票名字)。把很多只股票的价格 Series 并排拼在一起,共享同一套日期索引,就组成了一张 DataFrame。这正是我们今天要造的「上百只票行情表」— 行是交易日,列是股票名。

那它跟昨天的 NumPy 数组什么关系?DataFrame 内部存数据用的就是 NumPy 数组,所以 NumPy 的矢量化速度它全继承了。区别在于,NumPy 数组只能用「第几行第几列」这种位置来取数,而 DataFrame 多了一层标签,你可以直接用「2024 年 3 月 1 号这天的茅台价格」这种人话来取数,不用费劲去数它在第几行。这层标签看似只是方便,实际上是后面所有高级功能(自动对齐、分组、时间重采样)的地基。

一句话:Series 是带标签的一列,DataFrame 是带行列标签的一张表,底层是 NumPy、表面是标签 — 既有速度又有可读性,这就是 Pandas 成为量化第一工具的原因。

CONCEPT_02

自动对齐 — Pandas 区别于 Excel 的杀手锏

这是今天最重要的概念,也是 Pandas 最神奇的能力。所谓自动对齐,就是当你对两个 Series 或两张 DataFrame 做运算时,Pandas 不是傻乎乎地按位置一一对应,而是先看双方的标签(index),把标签相同的数据对齐了再算,标签对不上的地方,结果自动标成 NaN(缺失值)。

举个最具体的例子。茅台的价格 Series 索引是它的交易日,五粮液的价格 Series 索引是它自己的交易日。假设某天茅台停牌、五粮液正常交易,这两个 Series 的日期就不完全一样。当你写茅台加五粮液时,Pandas 会逐个日期去配:两边都有的日期,正常相加;只有一边有的日期(比如茅台停牌那天),结果就是 NaN。你完全不用操心日期对不对得上,Pandas 替你在这件最容易出错的事自动做了。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

loc 和 iloc — 按标签取 vs 按位置取

选数据是 Pandas 里最高频的操作,而 loc 和 iloc 是这件事的两大工具,也是新手最容易搞混的一对兄弟,务必分清。

loc 是「按标签(label)选」。你给它的是真实的标签名字:日期、列名。比如 `df.loc['2024-03-01', '茅台']` 取的是「2024 年 3 月 1 号这天茅台的值」。它符合人的直觉 — 你想取哪天哪只,就报哪天哪只的名字。

iloc 是「按位置(integer location)选」。你给它的是纯数字位置,从 0 开始数。比如 `df.iloc[0, 0]` 取的是「第一行第一列」,不管这行这列的标签叫什么。当你不关心标签、只想按顺序取前几行后几列时用它。

这俩最坑的差别在「切片」上:loc 的切片包含右端点,iloc 的切片不包含右端点。这是从 Python 列表切片(不含右端点)继承又改了一半的设定。举

例:`df.loc['2024-03-01':'2024-03-31']` 会把 3 月 31 号这天也选进来(含右端点);而 `df.iloc[0:5]` 只选第 0 到第 4 行,第 5 行不选(不含右端点)。无数 bug 就出在这个「差一个」上 — 本以为取了 5 行结果取了 4 行,或者本以为取到月底结果少了一天。

记忆口诀:loc 跟标签走、含两端;iloc 跟位置走、不含尾。拿不准的时候,选完先看一眼 shape,确认行数列数对不对。

CONCEPT_04

dtype — 数据类型决定快慢和内存

dtype 是「数据类型」的缩写,指 DataFrame 每一列里装的是什麼类型的数据 — 是浮点数 float64、整数 int64、还是字符串 object。这东西看不见,却直接决定你的代码跑多快、吃多少内存,新手常常吃了亏都不知道为什么。

第一个坑是 object 类型。如果你从 CSV 或网页读数据,某一列本该是价格,却混进了一个非数字字符(比如某天写成了「停牌」两个字),整列就会被 Pandas 判定为 object(通用对象,实际存的是字符串)。object 列做运算时,Pandas 退化到逐个处理 Python 对象,慢得惊人,而且很多数学运算直接报错或给出莫名其妙的结果。所以拿到数据第一件事,就是 print 一下 dtypes,确认价格列是 float 而不是 object。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

apply 和 map — 把函数批量套到每列每行

学会了造表、对齐、选数,最后1块是「批量加工」。你经常需要对每一只股票做同样一件事:算它的年化收益、算它的最大回撤、打个行业标签。挨个写一遍显然不现实,这时就用 Pandas 的 apply、map、applymap 三件套。

apply 是主力。它能把你写的一个函数,自动套用到 DataFrame 的每一列(或每一行)。比如你写一个「算年化收益」的函数,df.apply(这个函数) 就会对 100 只股票每只都算一遍,返回 100 个结果,一行代码搞定 100 次运算。这就是「批量套函数」。map 是 Series 专用的版本,常用来做「值的映射」,比如把股票代码映射成行业名字。applymap 则是把函数套到 DataFrame 的每一个单元格(新版 Pandas 里改叫 map)。

但这里有个特别重要的提醒,跟昨天的矢量化思维一脉相承:apply 虽然方便,但它本质上还是在底层循环,速度远不如 NumPy 的矢量化操作。能用 df.mean()、df.std()、df.pct_change() 这种内置矢量化方法直接算的,千万别用 apply 套个自定义函数去算 — 那会慢几十倍。apply 的正确定位是:当你的逻辑复杂到没有现成的矢量化方法时,才用它兜底。比如今天我们用 apply 算年化收益(因为涉及连乘和开方,没有一步到位的内置方法),但算年化波动时就直接用矢量化的 std,不套 apply。

一句话:apply 是「批量套函数」的瑞士军刀,方便但偏慢;先找内置矢量化方法,找不到再用 apply 兜底。

04 · PYTHON LAB · 真实可跑代码

Pandas 核心五件套 — Series/ DataFrame / 自动对齐 / loc vs iloc / dtype 内存 / apply 批量因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_034_pandas_core.py - Pandas 核心:Series / DataFrame / 自动对齐 / loc vs  
iloc / dtype / apply  
# 用真实沪深 300 成分股造一张「上百只票」的行情 DataFrame,讲透 Pandas 五大基本功  
# 数据:baostock(免费、稳定、国内零翻墙)  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import baostock as bs  
  
pd.set_option('display.width', 140)  
pd.set_option('display.max_columns', 12)  
START = '2024-01-01'  
END = '2024-12-31'  
N_TARGET = 100 # 目标拉 100 只成分股  
  
# ===== 0. 数据拉取:沪深 300 成分股的日收盘价 =====  
print('==== 0. 数据拉取(baostock 沪深 300 成分)====')  
lg = bs.login()  
if lg.error_code != '0':  
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')  
  
# 先取沪深 300 成分股清单(代码 + 名字)  
rs = bs.query_hs300_stocks(date='2024-12-31')  
members = []  
while (rs.error_code == '0') and rs.next():
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/6)

Pandas 核心五件套 — Series/ DataFrame / 自动对齐 / loc vs iloc / dtype 内存 / apply 批量因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
row = rs.get_row_data() # [updateDate, code, code_name]
members.append((row[1], row[2]))
print(f'沪深 300 成分股清单共 {len(members)} 只,取前 {N_TARGET} 只做演示')
members = members[:N_TARGET]

def bs_close(code):
    """拉单只股票日线收盘价(前复权),返回 Series(index=日期)"""
    rs = bs.query_history_k_data_plus(
        code, 'date,close', start_date=START, end_date=END,
        frequency='d', adjustflag='2')
    rows = []
    while (rs.error_code == '0') and rs.next():
        rows.append(rs.get_row_data())
    if not rows:
        return None
    df = pd.DataFrame(rows, columns=['date', 'close'])
    df['date'] = pd.to_datetime(df['date'])
    s = df.set_index('date')['close'].astype(float)
    return s

# 铁律 39:多 ticker 用 dict 显式映射,绝不 df.columns = list(...) 赋值
series_map = {}
for i, (code, name) in enumerate(members, 1):
    s = bs_close(code)
    if s is not None and len(s) > 200: # 过滤当年新上市/停牌过多的
        series_map[name] = s
    if i % 20 == 0:
        print(f' 已拉 {i}/{len(members)} 只,有效 {len(series_map)} 只')
bs.logout()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/6)

Pandas 核心五件套 — Series/ DataFrame / 自动对齐 / loc vs iloc / dtype 内存 / apply 批量因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 拼成一张大 DataFrame:行=交易日,列=股票名,自动按日期索引对齐
px = pd.DataFrame(series_map)
print(f'\n行情 DataFrame 形状 = {px.shape} (行={px.shape[0]} 个交易日, 列={px.shape[1]} 只股票)')

# ===== 1. Series:带标签的一维数组 =====
print('\n==== 1. Series:带标签的一维数组 ====')
one = px.iloc[:, 0] # 取第一只股票的价格 Series
print(f'第一只股票 = {one.name}')
print(f'类型 = {type(one).__name__}, 长度 = {len(one)}, dtype = {one.dtype}')
print(f'索引类型 = {type(one.index).__name__} (这就是「带日期标签」)')
# 按「标签」取值:用日期字符串,不用记第几行
print('按日期标签取值 one.loc["2024-03-01":"2024-03-05"]:')
print(one.loc['2024-03-01':'2024-03-05'])
print(f'描述统计:均值 {one.mean():.2f} 最高 {one.max():.2f} 最低 {one.min():.2f}')

# ===== 2. dtype:数据类型决定速度与内存 =====
print('\n==== 2. dtype:类型决定速度与内存 ====')
print('DataFrame 各列 dtype(应全是 float64):')
print(px.dtypes.value_counts())
mem_f64 = px.memory_usage(deep=True).sum() / 1024
px_f32 = px.astype('float32')
mem_f32 = px_f32.memory_usage(deep=True).sum() / 1024
print(f'float64 占内存 {mem_f64:.0f} KB')
print(f'float32 占内存 {mem_f32:.0f} KB(省了 {(1-mem_f32/mem_f64)*100:.0f}%)')
# 整数除法截断的坑演示
print('整数除法的坑:5 // 2 =', 5 // 2, ' 但 5 / 2 =', 5 / 2)
print('所以金融数据创建时务必 dtype=float,避免整数截断')

# ===== 3. 自动对齐:Pandas 的杀手锏 =====
print('\n==== 3. 自动对齐 by index ====')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/6)

Pandas 核心五件套 — Series/ DataFrame / 自动对齐 / loc vs iloc / dtype 内存 / apply 批量因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
a = px.iloc[:, 0].copy() # 第一只,完整
b = px.iloc[:, 1].copy() # 第二只
b_gap = b.drop(b.index[10:15]) # 故意删掉中间 5 天,模拟两表日期不一致
summed = a + b_gap # 相加 → 按日期对齐,缺失的日期结果为 NaN
n_nan = int(summed.isna().sum())
print(f'a 长度 {len(a)},b_gap 长度 {len(b_gap)}(删了 5 天)')
print(f'a + b_gap 后 NaN 个数 = {n_nan}(正是 b_gap 缺的那几天 - 自动对齐的证据)')
print('教训:不用手工对齐日期,Pandas 按 index 自动对齐;但缺口会变 NaN,要主动 dropna/fillna')

# ===== 4. loc vs iloc:标签 vs 位置 =====
print('\n=== 4. loc(按标签) vs iloc(按位置) ===')
by_label = px.loc['2024-03-01':'2024-03-31'] # 标签切片:含右端点 3-31
by_pos = px.iloc[0:5, 0:3] # 位置切片:不含右端点(0~4 行, 0~2 列)
print(f'loc["2024-03-01":"2024-03-31"] 选出 {by_label.shape[0]} 行(标签切片含两端)')
print(f'iloc[0:5, 0:3] 选出 {by_pos.shape[0]} 行 {by_pos.shape[1]} 列(位置切片不含右端点)')
print('记牢:loc 含右端点,iloc 不含 - 新手最爱在这里差一个')

# ===== 5. apply / map:把函数批量套到每列 =====
print('\n=== 5. apply 批量算因子 ===')
rets = px.pct_change().dropna(how='all')
def ann_return(col):
    """对一列日收益算年化收益率"""
    col = col.dropna()
    if len(col) < 2:
        return np.nan
    total = (1 + col).prod()
    return total ** (252 / len(col)) - 1
ann = rets.apply(ann_return) # 对每一列(每只股票)套用
ann_vol = rets.std() * np.sqrt(252) # 年化波动(矢量化,比 apply 快)
print(f'apply 算出 {len(ann)} 只股票的年化收益')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/6)

Pandas 核心五件套 — Series/ DataFrame / 自动对齐 / loc vs iloc / dtype 内存 / apply 批量因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('年化收益 最高 5 只:')
print((ann.sort_values(ascending=False).head() * 100).round(1).astype(str) + '%')
print('年化收益 最低 5 只:')
print((ann.sort_values().head() * 100).round(1).astype(str) + '%')
print(f'全部股票 年化收益中位数 = {ann.median()*100:.1f}%, 正收益占比  
{(ann>0).mean()*100:.0f}%')

# ===== 6. 出图 =====
# chart_01:100 只票价格面板热图 — 一张「大表」长什么样 + 市场齐涨齐跌
counts = px.notna().sum()
print(f'数据完整度:{(counts==px.shape[0]).sum()}/{px.shape[1]} 只全勤(满格  
{px.shape[0]} 天), 最少的也有 {counts.min()} 天')
norm = px / px.iloc[0] # 每只票按首日归一到 1, 才能放同一张图比较
best = norm.iloc[-1].idxmax(); worst = norm.iloc[-1].idxmin()
print(f'全年最强 {best} 收于首日的 {norm[best].iloc[-1]:.2f} 倍, 最弱 {worst} 仅  
{norm[worst].iloc[-1]:.2f} 倍')
# 找全市场齐涨最猛的一天(当天上涨股票占比最高)
up_ratio = (px.pct_change() > 0).mean(axis=1)
rally_day = up_ratio.idxmax()
print(f'全市场最齐涨的一天 = {rally_day.date()}(当天 {up_ratio.max()*100:.0f}% 的股票  
上涨)')
fig, ax = plt.subplots(figsize=(12, 6))
im = ax.imshow(norm.T.values, aspect='auto', cmap='RdYlGn_r', vmin=0.6, vmax=1.6,
extent=[0, px.shape[0], px.shape[1], 0])
ax.set_xlabel('交易日序号(2024 全年, 约 242 天)'); ax.set_ylabel('股票(100 只, 每行一  
只)')
ax.set_title('100 只沪深 300 成分股 2024 价格面板(每只首日归一 · 红涨绿跌)')
fig.colorbar(im, ax=ax, fraction=0.025, pad=0.02, label='相对首日净值')
fig.tight_layout(); fig.savefig('chart_panel.png', dpi=130); plt.close(fig)

# chart_02:年化收益分布直方图(apply 的成果)
fig, ax = plt.subplots(figsize=(10, 5))
```

```
ax.hist(ann.dropna().values * 100, bins=25, color='#2ca02c', edgecolor='white')
ax.axvline(0, color='#d62728', ls='--', lw=1.5, label='零线')
ax.axvline(ann.median()*100, color='black', ls=':', lw=1.5, label='中位数')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/6)

Pandas 核心五件套 — Series/ DataFrame / 自动对齐 / loc vs iloc / dtype 内存 / apply 批量因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.set_xlabel('2024 年化收益率 (%)'); ax.set_ylabel('股票只数')
ax.set_title(f'{px.shape[1]} 只沪深 300 成分股 2024 年化收益分布')
ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig('chart_ann_return.png', dpi=130); plt.close(fig)

# chart_03: dtype 内存对比
fig, ax = plt.subplots(figsize=(7, 5))
ax.bar(['float64', 'float32'], [mem_f64, mem_f32], color=['#1f77b4', '#ff7f0e'])
for i, v in enumerate([mem_f64, mem_f32]):
    ax.text(i, v, f'{v:.0f} KB', ha='center', va='bottom')
ax.set_ylabel('内存占用 (KB)'); ax.set_title('同一张行情表: float64 vs float32 内存对比')
ax.grid(alpha=0.3, axis='y')
fig.tight_layout(); fig.savefig('chart_dtype_mem.png', dpi=130); plt.close(fig)

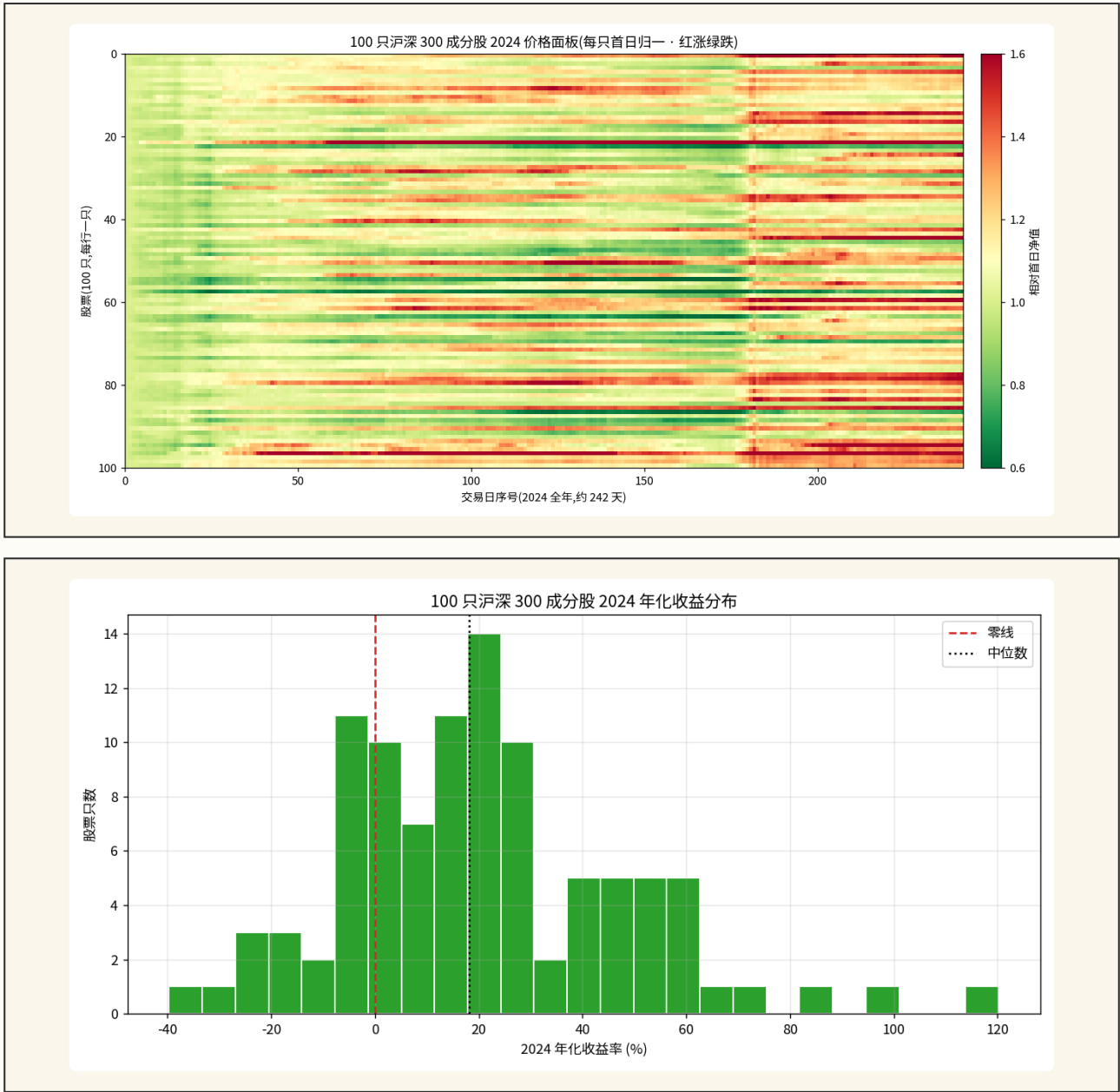
# chart_04: loc 标签切片选出的一段行情(头 3 只票, 3 月)
seg = px.loc['2024-03-01':'2024-03-31'].iloc[:, :3]
seg_norm = seg / seg.iloc[0] # 归一到 1, 方便同图比较
fig, ax = plt.subplots(figsize=(10, 5))
for c in seg_norm.columns:
    ax.plot(seg_norm.index, seg_norm[c], label=c, lw=1.8)
ax.set_title('loc 按日期标签切出 2024 年 3 月行情(归一化)')
ax.set_ylabel('相对净值'); ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig('chart_loc_slice.png', dpi=130); plt.close(fig)

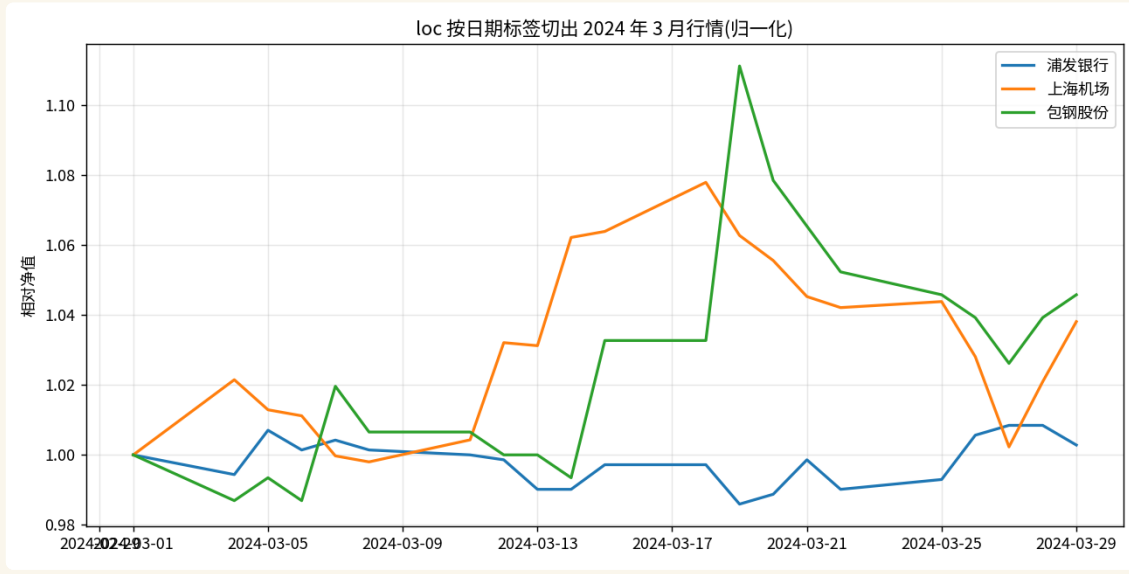
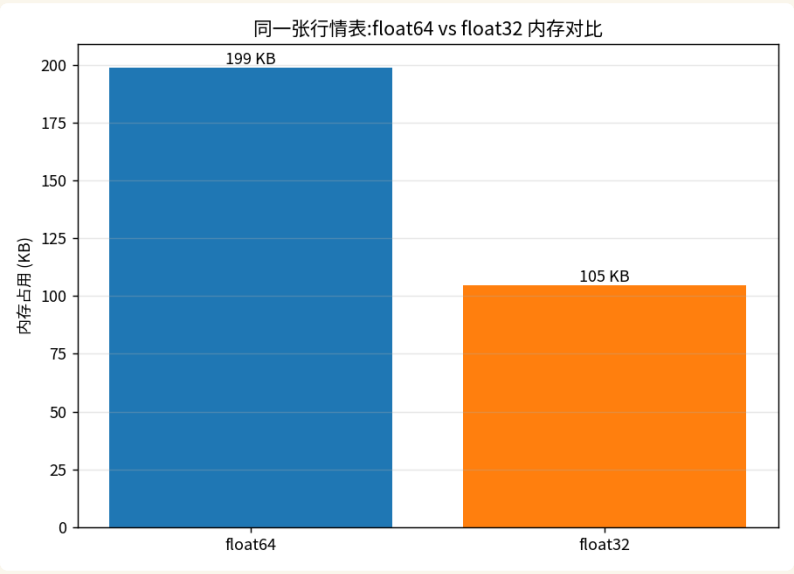
print('\n[done] 4 张图已保存,output.txt 见上方全部打印')
```

05 · CHART + ANALYSIS · 看图说话

回测结果

// · matplotlib 真实输出





行情大表规模

沪深 300 前
100 只成分股
× 2024 全年
242 个交易日
= 一张 242
行 100 列的
DataFrame,
100 只全部满
格 242 天

全市场最齐涨的一天

2024-09-30,
当天 99% 的
股票一起上涨
— 热图上一条
贯穿所有股票
的红色竖带

全年最强 vs 最弱

最强宇通客车
收于首日的
2.13 倍(涨
113%),最弱用
友网络仅
0.62 倍(跌
38%),同一年
分化巨大

apply 批量算年化收益

100 只年化收
益中位数
18.2%,正收益
占比 77%;前
五:宇通客车
120.2% / 小
商品城
94.9% / 赛
力斯
87.4% / 福
耀玻璃
74.2% / 浦
发银行 65.2%

dtype 内存对比

同一张表
float64 占
199 KB,转
float32 占
105 KB,省了
47%(近一半)

自动对齐产生 NaN

两个 Series
一个删掉中间
5 天,相加后
精确产生 5
个 NaN,正是缺
失的那几天 —
按 index 对
齐的铁证

ANALYSIS · 这张图告诉我们什么

► 01 9-30 当天 99% 齐涨:市场因子的肉眼版红带

2024-09-30 当天 99% 的股票一起上涨,在价格面板热图上是一条贯穿全部 100 只的红色竖带 — 这正是 D33 讲的『市场因子』的肉眼版:一只看不见的手同时拉动所有股票,行业差别在系统性行情前几乎消失。

► 02 100 只全勤干净:正说明真实研究要 dropna

100 只成分股全年满格 242 天、数据干净,是因为选的都是老牌大盘股;真实研究里新上市、退市、长期停牌的股票会留下缺口,这正是『自动对齐产生 NaN、必须主动 dropna』在真实数据里的意义。

► 03 中位数 18.2%、正收益 77%,但个股分化巨大

同样一年,宇通客车涨 113%、用友网络跌 38%,100 只年化收益中位数 18.2%、正收益 77% — 2024 年(尤其下半年 930 行情后)整体偏强但个股分化极大,印证『指数涨不代表你买的票涨』。

► 04 **float32 省 47% 内存,价格收益精度完全够用**

float32 比 float64 省 47% 内存而精度对价格收益完全够用 — 几百只票几年日线差别不大,但放到几千只票的分钟数据就是几个 G 的省与不省,是实盘工程最朴素的优化。

► 05 **删 5 天精确得 5 个 NaN:按标签对齐杜绝错位**

删掉 5 天后相加精确得到 5 个 NaN — 自动对齐严格按日期标签配对,不会像 Excel 那样物理错位,这是 Pandas 处理金融数据最可靠的底层保障。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

A 股散户

茅台 / 五粮液 等白酒板块

散户想验证两只同板块龙头是否同步,用 Excel 手工导出粘贴两列价格算相关系数,因日期错位算出离谱的低相关。换成 Pandas,两个价格 Series 自带日期索引,自动对齐后算相关系数,结果立刻正常。这正是 Pandas 自动对齐相对 Excel 手工党的降维打击,也是这一节故事的原型。

量化研究员日常

沪深 300 / 中证 500 全成分

研究员每天面对的不是一只股票,而是几百只股票的行情大表,本质就是一张大 DataFrame:行是交易日、列是股票代码。取某段时间窗口用 loc 按日期切片,取前几只看一眼用 iloc 按位置切片,算每只股票的因子用 apply 批量套函数 — 全是今天学的基本功,熟练度直接决定研究效率。

高频 / 实盘工程

全市场分钟级行情库

实盘系统要把几千只股票多年的分钟数据装进内存。同样一张表,用 float64 可能占好几个 G、内存吃紧,改成 float32 内存直接砍半,同样的服务器能多装一倍数据,响应也更快。精度损失对价格收益类数据可以忽略。dtype 优化是实盘工程里最朴素也最有效的省钱手段之一。

回测引擎

任意标的的历史区间

回测时要反复截取「某段时间窗口」的数据喂给策略,比如只回测 2024 年 3 月这一段。用 `df.loc['2024-03-01':'2024-03-31']` 一行按日期标签切出来,含起止两天,直观又不易错。如果误用 `iloc` 按位置切,既要先数清位置、又会漏掉右端点那一天,这是回测里很常见的「少一天」bug 来源。

因子计算

全市场选股池

构建选股因子时,要对池子里每只股票算同一个指标(年化收益、波动率、动量等)。用 `df.apply(因子函数)` 一次性对所有股票算完,返回一个排序用的因子值序列。但老手会优先用内置矢量化方法(`mean/std/pct_change`)算,只有逻辑复杂时才退而用 `apply`,因为 `apply` 在几千只股票上会明显变慢。

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

loc 和 iloc 切片端点搞混 — 差一行差一天

loc 切片含右端点,iloc 切片不含右端点。新手经常 `iloc[0:5]` 以为取了 5 行(确实是 5 行,0 到 4),却又在 `loc['2024-01-01':'2024-12-31']` 时忘了它会把 12 月 31 号也算进来,导致跟预期差一个。**正确做法**:记牢「loc 含两端、iloc 不含尾」,切完立刻看 `shape` 核对行列数,别凭感觉。

△ 02

不看 dtype 就运算 — 价格被当字符串

从 CSV 或接口读来的数据,价格列可能因为混进个别非数字字符而整列变成 object(字符串)。这时做加减乘除要么报错、要么字符串拼接出怪结果,而且慢得惊人。**正确做法**:数据一进门先 `print(df.dtypes)`,确认数值列是 `float64`;不是就用 `pd.to_numeric` 或 `astype(float)` 转,转的时候注意处理那些脏字符。

△ 03

忽视自动对齐产生的 NaN — 结果莫名其妙全空

两个索引差别很大的 Series/DataFrame 相加,自动对齐会让大量对不上的位置变成 NaN,严重时结果几乎全空,而很多后续运算碰到 NaN 会直接传染(NaN 加任何数还是 NaN)。**正确做法**:做完跨表运算后立刻检查 `.isna().sum()`,该 `dropna` 的 `dropna`、该 `fillna` 的 `fillna`,主动管理缺失值,别让 NaN 悄悄毁掉结果。

△ 04

链式索引引发 SettingWithCopyWarning — 改了个寂寞

写 `df[df['价格']>100]['仓位'] = 1` 这种「先筛选再赋值」的链式写法,Pandas 可能改在一个临时拷贝上,原表纹丝不动,还弹一个 `SettingWithCopyWarning` 警告。新手常常忽略这个警告,以为改成功了。**正确做法**:用 `df.loc[筛选条件, '列名'] = 值` 这种 `loc` 一步到位的写法赋值,既不会改到拷贝,也不会报警告。

△ 05

滥用 apply 当矢量化用 — 慢几十倍

`apply` 方便,但它本质是底层循环,远慢于 NumPy 矢量化。把本可以用 `df.mean()`、`df.pct_change()` 一步算的东西硬套个 `apply` 自定义函数,在几千只股票上会慢几十倍,白白浪费时间。**正确做法**:先找有没有现成的内置矢量化方法,找到就用;只有逻辑复杂、没有现成方法时,才用 `apply` 兜底。这是昨天矢量化思维的延续。

Pandas 核心实战 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01** 数据一进门先 `print(df.shape)` 和 `df.dtypes` — 确认行列数对、价格列是 `float64` 不是 `object`
- 02** 选数据分清 `loc` 和 `iloc` — 按日期/列名用 `loc`(含两端),按位置用 `iloc`(不含尾),切完看 `shape` 核对
- 03** 跨表/跨 Series 运算后立刻查 `.isna().sum()` — 自动对齐会制造 `NaN`,主动 `dropna` 或 `fillna`
- 04** 赋值用 `df.loc[条件, 列] = 值` 一步到位 — 绝不用链式 `df[...][...] = 值`(会改到拷贝且报警告)
- 05** 内存紧张时数值列转 `float32` — 价格收益类数据精度足够,内存直接砍半
- 06** 批量加工先找内置矢量化方法 — `mean/std/pct_change` 能算的别套 `apply`,`apply` 是逻辑复杂时的兜底
- 07** 造多列 DataFrame 用 `dict` 显式映射 {名字: Series} — 绝不用 `df.columns = list(...)` 赋值(列顺序会错位)

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 Series 是带标签的一列、DataFrame 是带行列标签的整张表 — 底层是 NumPy 数组, 表面多了一层索引标签, 既快又可读
- 02 自动对齐是 Pandas 杀手锏 — 运算时按索引对齐而非按位置, 日期对不上处填 NaN, 从根上杜绝 Excel 的错位 bug
- 03 自动对齐的影子是 NaN — 跨表运算后要主动 `.isna().sum()` 检查, 该 `dropna` 该 `fillna`
- 04 `loc` 按标签选含两端、`iloc` 按位置选不含尾 — 这是高频「差一个」bug 的来源, 切完看 `shape` 核对
- 05 `dtype` 决定快慢和内存 — `object` 慢、价格必须 `float`; `float32` 比 `float64` 省一半内存; 整数除法会截断, 创建时显式 `float`
- 06 `apply` 是批量套函数的瑞士军刀 — 方便但偏慢(本质循环), 能矢量化就别用 `apply`, 它是逻辑复杂时的兜底
- 07 造表用 `dict` 显式映射 — {股票名: 价格 Series} 拼成 DataFrame, 绝不用 `columns` 赋值(会列错位, 昨天的铁律延续)

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 Series 和 DataFrame 分别是什么?它们跟昨天学的 NumPy 数组是什么关系?(提示:Series 带标签的一列、DataFrame 带行列标签的表;底层都是 NumPy 数组加索引)

Q02 什么是 Pandas 的自动对齐?它怎么从根上避免了开头小张用 Excel 算出离谱相关系数的错误?(提示:运算按 index 对齐而非按位置;日期对不上处自动填 NaN,不会错位)

Q03 loc 和 iloc 有什么区别?df.loc['2024-01-01':'2024-12-31'] 和 df.iloc[0:5] 各自选出什么?(提示:loc 按标签含两端、iloc 按位置不含尾;前者含 12-31、后者只到第 4 行)

Q04 为什么要关注 dtype?一列价格的 dtype 是 object 会有什么后果?float32 相比 float64 有什么取舍?(提示:object 是字符串、运算慢/报错;float32 省一半内存、精度略低但够用)

Q05 apply 是干什么的?为什么说「能矢量化就别用 apply」?什么时候才该用 apply?(提示:批量把函数套到每列;apply 本质循环偏慢;只有没有现成矢量化方法、逻辑复杂时才用)

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Wes McKinney 《Python for Data Analysis》(2022 第 3 版, O'Reilly)– 第 5 章专讲 Pandas 入门、第 8 章讲数据对齐与合并, Pandas 作者亲笔, 绝对的标准教材
- ▶ Jake VanderPlas 《Python Data Science Handbook》(2016, O'Reilly, jakevdp.github.io 免费在线)– 第 3 章 Data Manipulation with Pandas, Series/DataFrame/索引/对齐讲得清晰易懂
- ▶ Pandas 官方文档《10 Minutes to pandas》(免费在线, pandas.pydata.org)– 官方快速入门, 例子可直接复制运行, 有问题先查官方文档最准
- ▶ Pandas 官方《Indexing and selecting data》(免费在线)– loc/iloc/链式索引/SettingWithCopyWarning 的权威说明, 把今天的高频坑讲到底
- ▶ Python 工具栈 – Pandas(核心)、pyarrow(更快的字符串/类型后端)、polars(Rust 写的高速替代, 语法相近)、pandas-ta(技术指标库), 这四个是表格数据处理的进阶生态, P3 阶段会用到

NEXT LECTURE · 下一节预告

DAY 035 时间索引与重采样

第 35 课 时间索引与重采样。今天我们的 DataFrame 行标签是日期, 但只是把日期当普通标签用。明天要把日期玩出花来 — 讲 DatetimeIndex 这个专门的时间索引, 讲 resample 怎么把分钟数据聚合成日线、把日线聚合成周线月线, 还有怎么处理时区、怎么按工作日偏移。时间是金融数据的灵魂, 学会了时间索引, 你才算真正掌握了 Pandas 处理行情数据的看家本领。